

Codebase Change Summary — Week 3

Equivalence Partitioning, Boundary-Value Analysis, and Negative Testing

SE 413/513 — Software Engineering

Demo Testing Suite

What changed in `demo-testing-suite/` since Week 2, and why

1 Purpose of This Document

The `demo-testing-suite/` codebase is a single, shared project that grows one lecture at a time rather than being rebuilt from scratch each week. This document summarizes exactly what was added or changed for Week 3 and, more importantly, explains *why* each change exists — what testing concept from the Week 3 reading it is meant to demonstrate. Read this alongside the reading and `demo-testing-suite/README.md`.

Key Idea

Nothing from Weeks 1–2 was removed or rewritten. Everything below is *additive* — the running examples you already saw (`calculate_late_fee`, the shopping cart, discounts) are all still there, unchanged, and still behave the same way.

2 What Changed At a Glance

File	Status	Purpose
<code>app/pricing/shipping.py</code>	New	The Week 3 worked example: a weight-based shipping-cost function with equivalence classes, boundaries, and a deliberate defect for you to find.
<code>tests/pricing/test_shipping.py</code>	New	A parametrized <code>pytest</code> suite encoding equivalence partitioning, boundary-value analysis, and negative testing against that function.
<code>docs/DEMO_GUIDE_week03_ep_bva_negative_testing.md</code>	New	A section lecture companion (primarily for in-class use).
<code>README.md</code>	Modified	Updated project layout description and the expected <code>pytest -v</code> output, now that Week 3's files exist.

3 New App Code: `determine_shipping_cost()`

Location: `app/pricing/shipping.py`

This is the centerpiece example for this lecture. Its specification (in the file's own docstring) is:

Given a package weight in kilograms, return the shipping cost: \$5.00 for $0 < weight \leq 1$ kg, \$10.00 for $1 < weight \leq 5$ kg, and \$20.00 for $5 < weight \leq 20$ kg. Weights of 0 kg or less, or above 20 kg, are invalid and should raise a `ValueError`.

This specification is deliberately identical in shape to the `determine_shipping_cost()` worked example walked through step by step in the Week 3 reading — so you can use the reading's equivalence-class table and boundary table directly against this real file instead of against a hypothetical one.

Common Pitfall

This implementation is not guaranteed to be correct. It contains at least one defect placed there on purpose, at one of the specification's boundaries. Finding it is meant to be an exercise in applying boundary-value analysis yourself — work out the boundary values from the specification first, predict the expected cost at each one, *then* check what the function actually returns. This document intentionally does not name the exact defect; treat that as part of the exercise.

There is also a second, separate issue in how the function responds to input of the wrong *type* entirely (not just the wrong numeric range) — worth investigating once you've finished with the numeric boundaries.

4 New Tests: `test_shipping.py`

Location: `tests/pricing/test_shipping.py`

This file encodes the three techniques from the reading as one parametrized `pytest` suite, in the same order the reading builds them:

- **Equivalence partitioning:** one representative value from each valid class (below the first boundary, between each pair of boundaries, above the last valid boundary).
- **Boundary-value analysis:** the exact edge of every class in the specification, tested together with the equivalence-partitioning cases in one parametrized test function.
- **Negative testing:** two separate test functions — one supplying out-of-range numeric weights and asserting a `ValueError`, and one supplying values of the wrong type entirely (`None`, a string, a list, a dictionary) and asserting some kind of clear error.

Tip

Run the suite yourself before reading further:

```
cd demo-testing-suite
pytest tests/pricing/test_shipping.py -v
```

One case in the output is marked XFAIL (“expected failure”) rather than PASS or FAIL. That is not a broken test — it is how this codebase documents a *known*, not-yet-fixed defect, exactly as `calculate_late_fee`’s negative-days bug was documented back in Week 1. Use the XFAIL line as a hint about *where* the boundary defect lives, without reading the reason string just yet if you want to work it out yourself first.

5 New Documentation

Location: docs/DEMO_GUIDE_week03_ep_bva_negative_testing.md

This is a lecture-companion document, primarily written to guide what gets shown and run during class, section by section. It is organized to follow the Week 3 reading’s own structure. You are welcome to read it, but it is not required reading — the reading itself and the exercise above are the primary material.

6 Updated: README.md

The top-level `demo-testing-suite/README.md` was updated to reflect the two new files above and to describe the suite’s current expected output. If you run the full suite from the project root:

```
cd demo-testing-suite
pip install -r requirements.txt
pytest -v
```

you should see a mix of passing tests, exactly two XFAIL results, and exactly one FAIL (all three explained in the README and the demo guide). If your output differs from that — more failures, fewer passes — something is broken and worth investigating or asking about; if it matches, everything is behaving as designed.

7 How This Connects to the Reading

Key Idea

The reading argues that equivalence partitioning gives you a small, representative set of inputs; boundary-value analysis tells you which values *within* that set are most worth testing; and negative testing checks that the software fails gracefully on input the specification never anticipated. `shipping.py` and `test_shipping.py` exist so you can apply all three techniques against a real function and a real, runnable test suite — not just a diagram on a slide.

8 Suggested Next Steps

1. Read `app/pricing/shipping.py`'s specification (the docstring), and independently derive its equivalence classes and boundary values, the same way the reading does for its own example.
2. Predict the expected output at every boundary value *before* running anything.
3. Run `pytest tests/pricing/test_shipping.py -v` and compare the actual results to your predictions. Investigate any mismatch.
4. Try at least one additional negative test of your own — an input category not already covered in the file — and reason about what the function *should* do with it, per the reading's guidance on graceful failure.